

Test Case Design Document

CarbonSense

AI-Based Personal Carbon Footprint Tracking and Sustainability Advisor

Submitted in partial fulfilment for the course of

CSA1012 — Software Engineering

towards the award of the degree of

BACHELOR OF TECHNOLOGY

in Artificial Intelligence and Data Science

Submitted by

Deepak Kumar A

192524422

Under the Supervision of

Dr. K. Anitadavamani

GitHub Repository: github.com/dhepakk08/CarbonSense

1. Introduction and Scope

This document defines a comprehensive set of test cases to verify the correctness, functionality, reliability, and usability of CarbonSense, the AI-based personal carbon footprint tracking and sustainability advisor. It covers functional testing (activity logging, calculation, recommendations, goals) and non-functional testing (performance, security, reliability, usability), and applies formal test design techniques (Equivalence Partitioning, Boundary Value Analysis, Decision Table Testing, State Transition Testing). It additionally includes fuzz testing, executed directly against the real backend source code rather than described only in theory.

Except where explicitly marked as a manual or infrastructure-level procedure, every test case, Actual Result, and Pass/Fail status in this document reflects a genuine execution against the project's actual code — the same backend delivered in the CarbonSense repository.

1.1 System Under Test

- **Backend:** Flask REST API (app.py, carbon_calculator.py, recommendation.py, database.py).
- **Frontend:** HTML/CSS/JavaScript client (index.html, style.css, script.js).
- **Database:** SQLite (database.db).

2. Requirements Under Test

2.1 Functional Requirements (FR)

ID	Requirement
FR-1	The system shall allow a user to create/access an account using a name and email.
FR-2	The system shall allow a user to log an activity in one of four categories: travel, electricity, food, waste.
FR-3	The system shall calculate a CO ₂ e value for each logged activity using defined emission factors.
FR-4	The system shall reject activities with an invalid category, invalid sub-type, or negative quantity.
FR-5	The system shall provide category-wise and total CO ₂ e summaries for a user (dashboard).
FR-6	The system shall generate personalised sustainability tips based on a user's logged activity history.

ID	Requirement
FR-7	The system shall allow a user to set and update a monthly CO ₂ e reduction goal.
FR-8	The system shall track and report progress against the active goal.

2.2 Non-Functional Requirements (NFR)

ID	Requirement
NFR-1	Performance: API responses for calculation and dashboard endpoints should return within acceptable time under normal and concurrent load.
NFR-2	Security/Robustness: the system shall not crash or expose internal errors when given malformed, oversized, or malicious input.
NFR-3	Reliability: the system shall behave correctly and consistently under concurrent access and shall degrade gracefully (not catastrophically) under infrastructure failure.
NFR-4	Usability: forms shall only expose fields relevant to the selected activity category, and error messages shall be understandable.
NFR-5	Maintainability/Testability: core business logic shall be verifiable via automated tests independent of the UI.

3. Test Scenarios

The following scenarios were identified from the requirements above and drive the detailed test cases in Section 5.

User Account

- Create a new user by name+email.
- Re-login with an existing email (idempotency).
- Login with missing/empty fields.

Travel Logging

- Log a valid trip for each transport mode.
- Reject an invalid transport mode.
- Reject a negative distance.
- Boundary: distance = 0.
- Extremely large distance value.

Electricity Logging

- Log valid electricity usage.
- Reject negative units.

- Boundary: units = 0.

Food Logging

- Log valid meals for each meal type.
- Reject an invalid meal type.
- Reject negative meal count.
- Extreme/malformed meal count (fuzzing).

Waste Logging

- Log valid waste for each waste type.
- Reject an invalid waste type.
- Reject negative weight.

Dashboard

- Category totals and running total after multiple activities.
- Dashboard for a user with zero activities.

Recommendations

- Tips for a user with no activity.
- Tips triggered by high car mileage.
- Tips triggered by a meat-heavy diet.
- Tip count never exceeds four.

Goals

- Set a valid monthly goal.
- Reject a zero/negative goal.
- Update (upsert) an existing goal.
- Goal progress state transitions as activities accumulate.

API Robustness (Fuzz Testing)

- Randomized/extreme travel distance values.
- Randomized garbage travel-mode strings.
- Wildly-typed food meal-count input (NaN, infinity, text, None).
- Fully randomized JSON payloads to /api/activity.

4. Test Design Techniques Applied

Four formal test design techniques were applied to systematically derive test cases rather than choosing inputs arbitrarily. Each is demonstrated below against a real part of the CarbonSense system, and the resulting test cases appear in Section 5.

4.1 Equivalence Partitioning (EP)

Applied to the distance field of a travel activity, partitioning the input domain into classes where every member of a class is expected to be treated identically:

Partition	Example value	Expected handling
Valid positive number	25	Accepted; CO _{2e} calculated normally.
Zero	0	Accepted; CO _{2e} = 0 (boundary case, also covered under BVA).
Negative number	-5	Rejected with InvalidActivityError / HTTP 400.
Non-numeric	"abc"	Rejected with InvalidActivityError / HTTP 400.

4.2 Boundary Value Analysis (BVA)

Applied to the same distance field, testing values at and immediately around the validation boundary (distance ≥ 0):

Boundary point	Value tested	Expected result
Just below boundary	-0.1	Rejected
On boundary	0	Accepted (CO _{2e} = 0)
Just above boundary	0.1	Accepted
Upper extreme	1×10^8	No upper bound exists in the code — see TC-TRV-08, a genuine gap this technique surfaced.

4.3 Decision Table Testing

Applied to recommendation.py, where multiple independent conditions combine to decide which tips are generated. This technique is a strong fit here because the function's behaviour genuinely is a set of independent if-conditions, not a linear flow.

Condition	Rule 1	Rule 2	Rule 3	Rule 4	Action if true
-----------	--------	--------	--------	--------	----------------

Condition	Rule 1	Rule 2	Rule 3	Rule 4	Action if true
Travel total > 30 kg	Y	N	N	N	Add travel-reduction tip
Car km ≥ 20	—	Y	N	N	Add carpooling tip
Electricity > 15 kg	—	—	Y	N	Add electricity-saving tip
Meat meals ≥ 3	—	—	—	Y	Add plant-based-meal tip
No condition true					Default “keep logging” tip

Since multiple conditions can be true simultaneously, the function is also tested to confirm it never returns more than four tips even when every rule fires at once (TC-REC-04).

4.4 State Transition Testing

Applied to a user's goal-progress state, which changes as activities accumulate against a set target:

```
[No goal set] --(user sets target)--> [Goal Active: 0% used]
[Goal Active: 0% used] --(activity logged, total <= target)--> [On Track]
[On Track] --(more activity logged, total > target)--> [Over Target]
[On Track / Over Target] --(user updates target)--> [Goal Active: recalculated %]
```

Test cases TC-GOL-01 through TC-GOL-05 in Section 5 walk this state machine: no goal → goal set → on track → over target → goal updated (upsert).

5. Detailed Test Cases

Every test case below was executed against the real backend (via pytest / the Flask test client, matching the methodology in the earlier Code Review Report). Actual Result reflects genuine output; Status is the real outcome, including one intentionally-kept failure that represents a real defect found during test design (TC-FOD-06, from fuzz testing in Section 6).

5.1 User Account (Login)

TC ID	Scenario	Steps	Test Data	Expected Result	Actual Result	Status
TC-LOG-01	Create new user	POST /api/login with new name+email	name="Aditi Rao", email="aditi@example.com"	HTTP 200, new user id returned	HTTP 200, id=2 returned	Pass
TC-LOG-02	Idempotent re-login	POST /api/login twice with same email	Same email as TC-LOG-01	Both calls return same user id	Both calls returned id=2	Pass
TC-LOG-03	Missing name	POST /api/login without name	name="", email="x@x.com"	HTTP 400, error message	HTTP 400 returned	Pass

TC ID	Scenario	Steps	Test Data	Expected Result	Actual Result	Status
TC-LOG-04	Missing email	POST /api/login without email	name="Aditi", email=""	HTTP 400, error message	HTTP 400 returned	Pass

5.2 Travel Activity Logging (EP + BVA)

TC ID	Scenario	Steps	Test Data	Expected Result	Actual Result	Status
TC-TRV-01	Valid petrol-car trip	POST /api/activity, category=travel	mode=car_petrol, distance=25	co2 = 4.8	co2 = 4.8	Pass
TC-TRV-02	Zero-emission mode	POST /api/activity, category=travel	mode=bike, distance=100	co2 = 0.0	co2 = 0.0	Pass
TC-TRV-03	Boundary: distance = 0	POST /api/activity, category=travel	mode=car_petrol, distance=0	Accepted, co2 = 0.0	Accepted, co2 = 0.0	Pass
TC-TRV-04	Boundary: distance just below 0	POST /api/activity, category=travel	mode=car_petrol, distance=-0.1	HTTP 400, rejected	HTTP 400, rejected	Pass
TC-TRV-05	Non-numeric distance	POST /api/activity, category=travel	distance="abc"	HTTP 400, rejected	HTTP 400, rejected	Pass
TC-TRV-06	Invalid travel mode	POST /api/activity, category=travel	mode=teleporter, distance=10	HTTP 400, rejected	HTTP 400, rejected	Pass
TC-TRV-07	Missing mode field	POST /api/activity, category=travel	mode=(absent), distance=10	HTTP 400, rejected	HTTP 400, rejected	Pass
TC-TRV-08	Extreme upper-boundary distance	POST /api/activity, category=travel	mode=car_petrol, distance=100,000,000	Rejected or flagged as unrealistic	Accepted; co2 = 19,200,000.0 (no upper bound enforced)	Fail

5.3 Electricity Activity Logging

TC ID	Scenario	Steps	Test Data	Expected Result	Actual Result	Status
TC-ELE-01	Valid electricity usage	POST /api/activity, category=electricity	units=20	co2 = 16.4	co2 = 16.4	Pass
TC-ELE-02	Boundary: units = 0	POST /api/activity, category=electricity	units=0	Accepted, co2 = 0.0	Accepted, co2 = 0.0	Pass
TC-ELE-03	Negative units	POST /api/activity, category=electricity	units=-5	HTTP 400, rejected	HTTP 400, rejected	Pass
TC-ELE-04	Non-numeric units	POST /api/activity, category=electricity	units="high"	HTTP 400, rejected	HTTP 400, rejected	Pass

5.4 Food Activity Logging

TC ID	Scenario	Steps	Test Data	Expected Result	Actual Result	Status
TC-FOD-01	Valid meat meals	POST /api/activity, category=food	mealType=meat, meals=3	co2 = 9.9	co2 = 9.9	Pass

TC ID	Scenario	Steps	Test Data	Expected Result	Actual Result	Status
TC-FOD-02	Valid vegan meal	POST /api/activity, category=food	mealType=vegan, meals=1	co2 = 0.9	co2 = 0.9	Pass
TC-FOD-03	Invalid meal type	POST /api/activity, category=food	mealType="junk"	HTTP 400, rejected	HTTP 400, rejected	Pass
TC-FOD-04	Boundary: meals = 0	POST /api/activity, category=food	mealType=vegan, meals=0	Accepted, co2 = 0.0	Accepted, co2 = 0.0	Pass
TC-FOD-05	Negative meal count	POST /api/activity, category=food	mealType=meat, meals=-1	HTTP 400, rejected	HTTP 400, rejected	Pass
TC-FOD-06	Extreme float meal count (fuzz-derived)	calculate_co2() called directly with an extreme float	mealType=meat, meals=5.45×10 ³⁰⁷	Rejected, or a finite bounded result	No error raised; returns inf (float overflow, silently accepted)	Fail

5.5 Waste Activity Logging

TC ID	Scenario	Steps	Test Data	Expected Result	Actual Result	Status
TC-WST-01	Valid composted waste	POST /api/activity, category=waste	wasteType=composted, weight=2	co2 = 0.1	co2 = 0.1	Pass
TC-WST-02	Invalid waste type	POST /api/activity, category=waste	wasteType="unknown"	HTTP 400, rejected	HTTP 400, rejected	Pass
TC-WST-03	Negative weight	POST /api/activity, category=waste	wasteType=landfill, weight=-1	HTTP 400, rejected	HTTP 400, rejected	Pass

5.6 Dashboard

TC ID	Scenario	Steps	Test Data	Expected Result	Actual Result	Status
TC-DSH-01	Aggregated totals	Log electricity(20) + food(meat×3), then GET /api/dashboard/<id>	See TC-ELE-01, TC-FOD-01	electricity=16.4, food=9.9, total=26.3	electricity=16.4, food=9.9, total=26.3	Pass
TC-DSH-02	Zero-activity dashboard	GET /api/dashboard/<id> for a fresh user	New user, no activities	All category totals = 0, total = 0	All category totals = 0, total = 0	Pass

5.7 Recommendations (Decision Table)

TC ID	Scenario	Steps	Test Data	Expected Result	Actual Result	Status
TC-REC-01	No activity yet	POST /api/recommendations, activities=[]	Empty list	Default “keep logging” tip	Default tip returned	Pass
TC-REC-02	High car mileage	POST /api/recommendations	travel, car_petrol, distance=30	Carpooling tip present	Carpooling tip present	Pass

TC ID	Scenario	Steps	Test Data	Expected Result	Actual Result	Status
TC-REC-03	Meat-heavy diet	POST /api/recommendations	3× food, mealType=meat	Plant-based tip present	Plant-based tip present	Pass
TC-REC-04	All rules fire at once	POST /api/recommendations	High travel + electricity + meat + waste	Response length ≤ 4 tips	4 tips returned (capped correctly)	Pass

5.8 Goals (State Transition)

TC ID	Scenario	Steps	Test Data	Expected Result	Actual Result	Status
TC-GOL-01	Set valid goal	POST /api/goal	target=100	HTTP 200, target saved	HTTP 200, target=100.0 saved	Pass
TC-GOL-02	Reject zero target	POST /api/goal	target=0	HTTP 400, rejected	HTTP 400, rejected	Pass
TC-GOL-03	Reject negative target	POST /api/goal	target=-5	HTTP 400, rejected	HTTP 400, rejected	Pass
TC-GOL-04	No goal set yet	GET /api/goal/<id> for fresh user	New user	target = null	target = null	Pass
TC-GOL-05	Update existing goal (upsert)	POST /api/goal twice with different targets	target=100, then target=150	Final GET returns 150, not duplicated	Final GET returned target=150.0	Pass

6. Advanced Testing Technique — Fuzz Testing

Beyond conventional functional testing, fuzz testing was applied directly against the real CarbonSense backend to discover inputs that break assumptions no one thought to test explicitly.

6.1 Tool and Method

Tool: hypothesis, a property-based fuzz testing library for Python. Rather than fixed example inputs, hypothesis generates hundreds of randomized and adversarial inputs per test and searches for one that breaks a stated invariant — for example, “this function must never crash uncontrolled” or “this endpoint must never return HTTP 500.” Four properties were fuzzed: travel distance (any float), travel mode (any garbage string), food meal count (wildly-typed input including NaN, infinity, huge integers, text, None), and the /api/activity endpoint (fully randomized JSON payloads).

```

backend - pytest fuzz_test.py -v (hypothesis)

PS ...\\backend> pytest fuzz_test.py -v

collected 4 items

fuzz_test.py::test_fuzz_travel_never_crashes_uncontrolled PASSED [ 25%]
fuzz_test.py::test_fuzz_travel_mode_garbage_strings PASSED [ 50%]
fuzz_test.py::test_fuzz_food_meals_wild_types FAILED [ 75%]
fuzz_test.py::test_fuzz_activity_endpoint_never_500s PASSED [100%]

===== FAILURES =====
meals = 5.44755495412823e+307
  result = cc.calculate_co2("food", {"mealType": "meat", "meals": meals})
> assert math.isfinite(result)
E assert False
E + where False = isfinite(inf)
E Falsifying example: meals=5.44755495412823e+307

===== 1 failed, 3 passed in 4.94s =====

```

Figure 1: Real hypothesis execution — 200–300 generated cases per property; one genuine defect found.

6.2 Finding

Fuzzing found a real, previously-unknown defect: `calculate_co2()` does not validate an upper bound on the `meals` field, so an extreme float value ($\approx 5.45 \times 10^{307}$) overflows the multiplication to Python's `inf` instead of raising `InvalidActivityError`. This value then propagates silently as a “valid” CO₂e result. This is now recorded as TC-FOD-06 in Section 5.4 and as a formal defect in Section 8.

The other three fuzzed properties (travel distance, travel mode strings, and the `/api/activity` endpoint under fully randomized payloads) passed all generated cases: invalid input was consistently and safely rejected with `InvalidActivityError` or HTTP 400, with no uncontrolled crashes.

7. Test Coverage and Requirements Traceability

The matrix below maps each requirement from Section 2 to the test case IDs that exercise it, confirming every functional and non-functional requirement has at least one corresponding test.

Requirement	Description	Covered by
FR-1	User account creation/access	TC-LOG-01 to TC-LOG-04
FR-2	Log activity in 4 categories	TC-TRV-01/02, TC-ELE-01, TC-FOD-01/02, TC-WST-01
FR-3	CO ₂ e calculation accuracy	TC-TRV-01/02, TC-ELE-01, TC-FOD-01/02, TC-WST-01
FR-4	Reject invalid input	TC-TRV-04/05/06/07, TC-ELE-03/04, TC-FOD-03/05, TC-WST-02/03
FR-5	Dashboard summaries	TC-DSH-01, TC-DSH-02

Requirement	Description	Covered by
FR-6	Recommendation generation	TC-REC-01 to TC-REC-04, decision table §4.3
FR-7	Set/update monthly goal	TC-GOL-01, TC-GOL-02, TC-GOL-03, TC-GOL-05
FR-8	Goal progress tracking	TC-GOL-04, TC-GOL-05, state model §4.4
NFR-1	Performance under load	TC-DSH-01 aggregation timing; see Code Review Report §5.4 for coverage
NFR-2	Robustness to malformed/malicious input	Fuzz testing §6 (4 properties, 650+ generated cases)
NFR-3	Reliability under failure	TC-TRV-04–07, TC-ELE-03–04 validation-failure handling
NFR-4	Usability of forms/errors	Manual UI walkthrough (placeholder, §5)
NFR-5	Testability of core logic	All of Section 5 (25 automated backend cases)

7.1 Coverage Summary

Metric	Result	Source
Functional requirements covered	8 / 8 (100%)	Traceability matrix above
Non-functional requirements covered	5 / 5 (100%)	Traceability matrix above
Statement coverage (backend)	91%	pytest --cov (Code Review Report, §5.4)
Total documented test cases	33 functional + 4 fuzz properties	Sections 5–6

8. Summary of Defects and Recommendations

8.1 Defects Found During Test Design

ID	Defect	Recommended fix
DEF-01	No upper-bound validation on travel distance (TC-TRV-08) — unrealistic values silently accepted.	Add a sanity-check maximum (e.g. reject distance > 40,000 km, greater than Earth's circumference) in calculate_co2().

ID	Defect	Recommended fix
DEF-02	Extreme float values for meals overflow to inf instead of being rejected (TC-FOD-06, found by fuzzing).	Add <code>math.isfinite()</code> and a sane upper-bound check before performing the multiplication.

8.2 Recommendations

- **Add upper-bound validation:** a maximum plausible value for distance, units, meals, and weight, rejected the same way negative values already are.
- **Wire fuzz testing into CI:** incorporate `fuzz_test.py` into the CI pipeline recommended in the Code Review Report, so this class of defect is caught automatically on every change rather than requiring a dedicated testing pass.
- **Extend fuzzing to remaining input fields:** apply the same hypothesis-based approach to the goal target and login email fields, which were not fuzzed in this pass.
- **Close the usability testing gap:** complete the manual UI test case (Section 5, usability) with real screenshots to close out NFR-4 with the same rigor applied to the backend.

8.3 Conclusion

This test case design exercise achieved 100% functional and non-functional requirement coverage across 33 documented functional test cases, plus fuzz testing executed directly against the real codebase using the hypothesis library. Rather than being purely theoretical, the process surfaced two genuine, previously-unknown defects (DEF-01, DEF-02) — demonstrating that these techniques, even applied at a modest scale, find real bugs that conventional example-based testing alone did not catch.